PROJECT REPORT

on

"PARKING MANAGEMENT SYSTEM"

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirements for the Award of

BACHELOR'S DEGREE IN
COMPUTER SCIENCE ENGINEERING
BY

ANKIT SEN 2305518
KUMAR SAMARTH 2305545
MRINMAY DAS 23051682
SHUBHAM CHAKRABORTY 2305572

UNDER THE SUPERVISION OF

DR. HIMADRI SHEKHAR ROY

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024

March 2026

CERTIFICATE

This is to certify that the project entitled

"PARKING MANAGEMENT SYSTEM"

has been successfully completed and submitted by:

ANKIT SEN	2305518
KUMAR SAMARTH	2305545
MRINMAY DAS	23051682
SHUBHAM CHAKRABORTY	2305572

The work presented in this report reflects original student effort undertaken in partial fulfilment of the Bachelor of Engineering programme in Computer Science Engineering at KIIT Deemed to be University, Bhubaneswar. The project was carried out during the academic session 2025-2026 under our direct guidance.

Date: ___/___/_____

(Dr. Himadri Shekhar Roy)
Project Supervisor

Acknowledgements

We take this opportunity to express our heartfelt gratitude to Dr. Himadri Shekhar Roy of the School of Computer Engineering, KIIT University, whose unwavering mentorship and constructive insights guided this project from its initial concept to its final form. His willingness to engage with our challenges and offer direction at critical junctures made a measurable difference to the quality of both the system and this report.

We are sincerely grateful to the faculty and administrative staff of the School of Computer Engineering for cultivating a learning environment that places real value on hands-on problem-solving. Their encouragement extended well beyond classroom instruction and contributed in meaningful ways to our growth as practising engineers.

We also wish to acknowledge our peers and batchmates who generously participated in the testing and evaluation of this project. Their candid feedback, drawn from actual usage rather than theoretical assumptions, helped us identify and address practical weaknesses that we might otherwise have overlooked.

Our deepest appreciation goes to our families, whose steady encouragement and patience kept us motivated throughout the duration of this work.

Signature of Group Member A: _____

Signature of Group Member B: _____

Signature of Group Member C: _____

Signature of Group Member D: _____

Abstract

As vehicle numbers on institutional campuses continue to climb, the limitations of manually administered parking systems have become increasingly difficult to ignore. Paper registers, handwritten entries, and error-prone billing routines place an unnecessary burden on staff while undermining the accuracy and accessibility of records. This project addresses that problem through the design and development of a web-based Parking Management System (PMS) tailored to the operational requirements of KIIT University.

The proposed solution automates the complete cycle of vehicle entry, duration computation, charge calculation, and data archival. Built on a client-server architecture, the frontend draws on HTML, CSS, and JavaScript to deliver a clean and intuitive operator experience, while a Python Flask server handles all backend processing and file persistence. Parking records are stored in an Excel workbook via the openpyxl library, a deliberate design choice that allows administrative staff to review and export data using tools they already encounter in their daily work.

The system supports three vehicle categories - bicycles, motorcycles, and cars - and three personnel designations: students, faculty, and staff. A login mechanism restricts access to authorised users, and a dedicated administrator panel enables comprehensive reporting with date-range filtering. The overall design prioritises ease of deployment, requiring no specialist infrastructure and minimal training to operate.

Keywords: Parking Management System, Flask, Python, Web Application, Excel Integration, Vehicle Tracking, Campus Automation, REST API

Contents

1	**Introduction**	1
2	**Basic Concepts / Literature Review**	3
2.1	HTML, CSS, and JavaScript	3
2.2	Python and the Flask Framework	3
2.3	The openpyxl Library	4
2.4	REST API Architecture	4
2.5	LocalStorage and Session Management	5
2.6	Literature Review	5
3	**Problem Statement / Requirement Specifications**	6
3.1	Project Planning	6
3.2	Project Analysis (SRS)	7
3.3	System Design	7
3.3.1	Design Constraints	7
3.3.2	System Architecture / Block Diagram	8
4	**Implementation**	10
4.1	Methodology / Proposal	10
4.2	Testing / Verification Plan	11
4.3	Result Analysis / Screenshots	12
4.4	Quality Assurance	13
5	**Standards Adopted**	13
5.1	Design Standards	13
5.2	Coding Standards	13
5.3	Testing Standards	14
6	**Conclusion and Future Scope**	15
6.1	Conclusion	15
6.2	Future Scope	16
7	**References**	17
8	**Individual Contributions**	18
9	**Plagiarism Report**	23

List of Figures

Figure	Caption	Page
3.1	System Architecture of the Parking Management System	12
3.2	Login Authentication Flowchart	13
3.3	Data Flow Diagram — Vehicle Entry to Storage	14

Chapter 1

Introduction

Vehicular congestion in institutional settings - particularly on university campuses - has grown into a significant administrative concern over the past decade. As the number of privately owned vehicles among students and faculty increases year on year, organisations that continue to rely on handwritten registers and manual billing are finding these approaches increasingly untenable. The human effort involved is substantial, the scope for error is considerable, and the absence of readily accessible records limits the institution's ability to make data-informed decisions about parking allocation and policy.

This project presents a Parking Management System (PMS) purpose-built for KIIT University that transitions the parking administration process from a paper-driven routine to a structured digital one. The system allows authorised operators to log vehicle arrivals and departures through a browser interface, with all duration computations and fee calculations handled automatically in accordance with the registered vehicle type. A searchable, date-filterable record of all transactions is maintained without any manual effort, providing both operational staff and institutional administrators with reliable, consistent data.

A deliberate goal of this project was to develop something genuinely practical rather than technically impressive in isolation. That meant avoiding dependencies on specialist hardware, commercial databases, or complex deployment procedures. The frontend was therefore built with standard HTML, CSS, and JavaScript, and the backend runs on Flask, a well-regarded Python web framework known for its simplicity. Data is stored in an Excel workbook managed through the openpyxl library - a format that any member of the administrative staff can open and navigate without technical assistance.

The system recognises three categories of vehicles - bicycles, motorcycles, and cars - each associated with a distinct hourly rate, and three types of personnel - students, faculty, and staff. Entry and departure times are recorded in twenty-four-hour format, and overnight parking is handled correctly through a dedicated edge-case correction in the calculation logic. An authentication gateway ensures only registered users may access the system, and a separate admin reporting view provides full oversight of the dataset.

The remainder of this report is organised as follows: Chapter 2 covers the technologies that underpin the system and situates the work within existing literature; Chapter 3 formalises the problem and specifies requirements alongside key architectural diagrams; Chapter 4 walks through the implementation and the testing regime; Chapter 5 describes the development and documentation standards applied; and Chapter 6 draws conclusions and identifies directions for future enhancement.

Chapter 2

Basic Concepts / Literature Review

This chapter introduces the technologies that form the technical foundation of the Parking Management System and reviews relevant prior work in the domain of campus parking digitisation.

2.1 HTML, CSS, and JavaScript

HyperText Markup Language (HTML) is the foundational language for defining the structure and content of web pages. In the context of this project, HTML was employed to construct every visible component of the user interface - the login screen, the vehicle data entry form, the navigational sidebar, and the tabular reporting view. Cascading Style Sheets (CSS) determine how those structural elements are rendered visually, governing layout, colour palette, typography, and spacing. Two dedicated stylesheets were produced: one for the primary application interface and another for the login page, together achieving a consistent, professional appearance suited to institutional deployment across varying screen sizes.

JavaScript operates within the browser environment to bring dynamic, interactive behaviour to otherwise static HTML. In this application, JavaScript manages the form submission process, computes parking duration and the corresponding charge in real time, maintains a live clock display, handles session-based authentication using browser storage, and enables seamless switching between the entry and reporting views without requiring a full page reload.

2.2 Python and the Flask Framework

Python is a widely adopted, high-level programming language appreciated for its legible syntax, strong community support, and rich ecosystem of libraries. It forms the basis for all server-side components in this project. Flask is a lightweight web framework written in Python that enables developers to build HTTP servers and RESTful application programming interfaces without the structural overhead that accompanies larger frameworks such as Django. Within the PMS, Flask manages two principal routes: one that receives parking records from the frontend and writes them to the spreadsheet, and another that serves the stored Excel file to the requesting client on demand.

The Flask-CORS extension was incorporated to address Cross-Origin Resource Sharing restrictions that browsers impose by default. This allows the browser-hosted frontend to interact freely with the Flask backend running on a local port, without the browser's built-in security policies interfering with the exchange of data.

2.3 The openpyxl Library

openpyxl is a dedicated Python library for creating, reading, and modifying Microsoft Excel workbooks in the standard .xlsx format. It fulfils the role of the persistence layer within this system. Each time the Flask backend receives a new parking submission, openpyxl appends the corresponding data as a fresh row in the parking_data.xlsx workbook. The library also handles the creation of the workbook itself and the initial setup of column headers when no prior data is present. The decision to use Excel as the data store - rather than a conventional database - was taken deliberately: it allows non-technical administrative staff to inspect, export, and work with the records directly in Microsoft Excel, with no requirement for a database server or specialist query knowledge.

2.4 REST API Architecture

Representational State Transfer, commonly referred to as REST, is an architectural approach to client-server communication over the HTTP protocol. It defines conventions for how resources should be addressed and how operations upon them should be expressed through standard HTTP methods. The PMS backend exposes two RESTful endpoints: the /submit route accepts HTTP POST requests carrying parking records encoded as JSON objects, and the /excel route responds to HTTP GET requests by streaming the current workbook file back to the caller as a downloadable attachment. This separation between writing and reading operations aligns with REST principles and ensures that the frontend and backend remain loosely coupled - each can be updated or replaced independently without breaking the interface between them.

2.5 LocalStorage and Browser-Based Session Management

User session state in this application is managed through the Web Storage API's localStorage facility, which provides persistent key-value storage within the browser. Upon a successful login, the authenticated username is saved to localStorage. Every subsequent page load inspects this stored value; if none is found, the visitor is automatically redirected back to the login screen. This approach provides a practical and adequate level of access control for the deployment context, without introducing the added complexity of maintaining server-side session records.

2.6 Literature Review

Scholarly and technical work in the area of campus parking automation has grown noticeably over the past several years, reflecting the increasing recognition that manual administration is not sustainable as vehicle counts rise. Earlier systems range from server-side PHP and MySQL applications to more contemporary implementations using NoSQL databases and cloud-based infrastructure. While the capabilities of these systems vary, a consistent finding across the literature is that transitioning from manual to digital record-

keeping reduces operator workload and substantially lowers the frequency of data entry errors.

Singla and Kaur (2020) document a PHP-MySQL web application for campus parking that achieved approximately a forty percent reduction in operator entry time compared with a paper-based baseline. Sharma, Verma, and Singh (2021) describe a more hardware-integrated solution combining RFID sensors with a Flask backend to automate vehicle detection at entry and exit points; while capable, such an approach demands significant infrastructure investment that is not always feasible. The present project occupies a practical middle ground: it delivers the efficiency gains of a digital system without requiring any dedicated hardware or database infrastructure, using a file-based approach that staff can work with independently using tools already part of their daily routine.

Chapter 3

Problem Statement / Requirement Specifications

Institutions with large student and faculty populations that manage vehicle parking through non-digital processes face a set of predictable and recurring problems. At KIIT University, the existing approach requires security and administrative personnel to record vehicle details by hand, estimate parking durations without any computational support, and maintain physical registers that are inherently susceptible to illegibility, loss, and physical damage over time. There is no systematic mechanism for generating summary reports, verifying billing accuracy after the fact, or querying past records by vehicle type or date. The cumulative result is a process that is prone to inconsistency, difficult to audit, and opaque to institutional management seeking reliable operational data.

The central objective of this project is to replace that ad-hoc manual process with a purpose-built digital system capable of capturing parking transactions accurately, computing fees without operator calculation, and retaining records in a format that is both accessible to non-technical staff and suitable for retrospective analysis.

3.1 Project Planning

Development of the Parking Management System was structured across five sequential phases:

1. Requirement Gathering: Identifying and agreeing on the specific data attributes to be captured for each parking transaction, including vehicle type, personnel classification, registration number, and entry and exit timestamps.
2. UI Design and Frontend Construction: Designing the operator interface in HTML, CSS, and JavaScript, with particular attention to clarity, minimal training requirements, and consistent feedback for every interaction.
3. Backend Development: Building the Flask server responsible for receiving, validating, and storing parking records, and for delivering stored data on request.
4. Integration and Functional Testing: Connecting the frontend and backend layers and executing a structured set of end-to-end test cases to verify that the full workflow performs as intended.
5. Review and Refinement: Addressing deficiencies and edge cases identified during testing prior to final submission.

The target feature set was established at the outset and encompassed a secure login interface, a vehicle data entry form with automatic computation, server-side persistence to an Excel workbook, and an administrator reporting module with date-range filtering.

3.2 Project Analysis (SRS)

A thorough analysis of the stated requirements produced the following set of design decisions and system constraints:

- The system would accommodate exactly three vehicle categories - Bicycle, Motorcycle, and Car - each assigned a distinct per-hour rate.
- Personnel designations would be limited to three options: Student, Faculty, and Staff.
- All times would be entered and stored in twenty-four-hour HH:MM format, eliminating ambiguity associated with twelve-hour notation.
- Overnight parking scenarios - where a vehicle departs after midnight relative to its recorded entry time - would be handled correctly through a dedicated correction in the calculation routine.
- Data persistence would be file-based (Excel workbook) to avoid dependency on a database engine and to keep deployment requirements minimal.
- System access would be governed by an authentication screen backed by a localStorage-managed session, redirecting unauthenticated visitors to the login page.

3.3 System Design

3.3.1 Design Constraints

The system was developed and tested on a Windows environment running Python 3.11. The full set of software components utilised is as follows:

- Python 3.11 - the server-side runtime environment.
- Flask 3.x and Flask-CORS - backend web framework and cross-origin request handler.
- openpyxl - library for constructing and modifying Excel workbooks.
- Modern web browser (Chrome, Edge, or Firefox) - client-side runtime for the frontend interface.
- Visual Studio Code - primary integrated development environment.

No external database engine was required at any stage of development or deployment. The Flask server is configured to listen on port 5000 of the host machine within a local network context, an arrangement that substantially simplifies the initial setup and eliminates the need for public server provisioning.

3.3.2 System Architecture (UML) / Block Diagram

The Parking Management System is built around a client-server model. The frontend - comprising HTML, CSS, and JavaScript - runs within the operator's web browser and communicates with the Flask backend via HTTP. When an operator submits a completed

entry form, the JavaScript layer constructs an HTTP POST request containing the form data serialised as a JSON payload, which is dispatched to the /submit route on the Flask server. The server validates the incoming data, appends it as a new row in parking_data.xlsx using openpyxl, and returns a JSON response confirming the successful write operation.

Figure 3.1 below illustrates the high-level architectural relationship between system components.

USER / OPERATOR (Browser)
↕ HTTP / JSON (REST API)
FLASK BACKEND SERVER (app.py)
↕ openpyxl (Read / Write)
DATA STORE (parking_data.xlsx)

Figure 3.1: System Architecture of the Parking Management System

Figure 3.2 presents the login authentication flowchart, detailing the decision logic that governs user access.

START
↓
User navigates to Login Page
↓
Enters Username and Password
↓
Credentials validated against stored values
YES → Store username in localStorage → Redirect to Dashboard
NO → Display inline error message (remain on Login Page)
↓
END

Figure 3.2: Login Authentication Flowchart

Figure 3.3 traces the complete data flow from operator input through front-end processing to final storage in the Excel workbook.

OPERATOR INPUT
Vehicle Type | Person Type | Registration Number | Entry/Exit Times
↓
PROCESSING (JavaScript)
Compute Duration & Charge → Construct JSON Payload
↓ HTTP POST /submit
FLASK BACKEND (app.py)
Validate Fields → Write Record via openpyxl
↓
DATA STORE (parking_data.xlsx)
Persistent record storage

Figure 3.3: Data Flow Diagram — Vehicle Entry to Storage

Chapter 4

Implementation

This chapter provides a detailed account of how each component of the Parking Management System was built, the procedures used to validate its behaviour, and the quality assurance practices observed throughout development.

4.1 Methodology / Proposal

The application is composed of three interconnected modules: the authentication module, the vehicle data entry module, and the backend data management module.

Authentication Module

The login screen (login.html) presents a minimal form requesting a username and password. All authentication logic resides in login.js. When the form is submitted, the supplied credentials are compared against a set of pre-defined constant values. A match causes the username to be written to localStorage and the user to be forwarded to the main application interface (index.html). An incorrect submission triggers an inline error notification without reloading the page, preserving the operator's working context and avoiding unnecessary delays.

Vehicle Data Entry Module

The entry form within index.html collects the following attributes for each parking transaction: the date, vehicle category (Bicycle, Motorcycle, or Car), personnel type (Student, Faculty, or Staff), passenger count, vehicle registration number, arrival time, departure time, any supplementary hours, total duration on site, and an optional remarks field. Total duration and the corresponding fee are derived automatically when the operator activates the Calculate button.

The calculation logic in script.js converts both timestamps from HH:MM notation to minutes elapsed since midnight. The departure value is subtracted from the arrival value, any supplementary hours are incorporated, and the resulting duration is multiplied by the rate assigned to the selected vehicle category. Rates are set at Rs. 5 per hour for bicycles, Rs. 10 for motorcycles, and Rs. 20 for cars. When a vehicle departs after midnight, the routine adds 1,440 minutes to the computed difference before proceeding, correctly resolving the overnight edge case.

Backend Data Management Module

The Flask application (app.py) exposes two HTTP endpoints. The /submit route receives POST requests carrying parking records as JSON, appends the corresponding data as a new row in parking_data.xlsx through openpyxl, persists the file to disk, and returns a JSON confirmation to the calling client. The /excel endpoint handles GET requests by transmitting the current workbook file to the browser, enabling administrators to access the complete dataset without any need to interact directly with the server's file system.

4.2 Testing / Verification Plan

A structured test suite was executed to confirm correct behaviour across standard operating conditions, boundary scenarios, and erroneous inputs. The outcomes of all test cases are documented in the table below.

ID	Test Case Title	Test Condition	System Behaviour	Expected Result
T01	Successful Login	Valid credentials submitted	Operator redirected to dashboard (index.html)	Login completes; username saved to session
T02	Failed Login	Incorrect password entered	Inline error message displayed on screen	Login rejected; no redirection occurs
T03	Charge Computation	Car parked 10:00–12:00 (2 hrs)	System displays 2 hrs, charges Rs. 40	Duration and cost accurately rendered
T04	Cross-Midnight Parking	Entry 22:00, exit 02:00	Duration computed as 4 hours via overnight logic	Overnight scenario handled without errors
T05	Complete Form Submission	All required fields populated and submitted	New row appended to parking_data.xlsx	Success confirmation shown; form resets
T06	Excel File Download	Administrator activates download link	File downloaded or opened in browser	File reflects the most recent parking entries
T07	Incomplete Submission	Form submitted without exit time	HTML5 browser validation activates	Submission blocked; required field highlighted
T08	User Logout	Logout button activated by operator	Session data cleared; login page displayed	Access denied without re-authentication

4.3 Result Analysis / Screenshots

All eight test cases were executed in full and produced outcomes consistent with their defined expectations. The fee computation logic handled both standard and overnight parking correctly across every trial. The HTML5 validation layer intercepted incomplete submissions before they reached the server in all cases, and each successful submission

was accurately reflected as a new row in the Excel workbook. The download endpoint returned the most recent version of the data file on every request without exception.

[Figure 4.1: Login Page — The login screen presents a compact form requesting a username and password. When an incorrect combination is entered, an inline error notification appears without triggering a page reload, preserving the operator's context.]

[Figure 4.2: Data Entry Form — The main interface displays all input fields alongside a real-time charge display. The sidebar permits navigation between the entry form and the reporting panel without a full page reload.]

[Figure 4.3: Reporting Panel — The admin view renders a tabular listing of all parking records. Records can be filtered by date range, and the full dataset can be downloaded as an Excel file from this panel.]

4.4 Quality Assurance

Quality was maintained throughout development through a combination of feature-level testing, integration verification, and scheduled peer review sessions within the team. Each new capability was exercised in isolation before being incorporated into the main codebase. Backend routes were validated independently by issuing direct fetch calls from the browser console and examining the resulting Excel file contents after each operation. Peer review sessions helped surface logical oversights and improved code legibility before the work was finalised for submission.

Chapter 5

Standards Adopted

5.1 Design Standards

The user interface was shaped by widely accepted web usability principles, drawing on Nielsen's usability heuristics as a reference framework. Every form field carries an explicit label, calculated outputs update in immediate response to user input, and the application provides visible, timely feedback for every significant interaction - including form submissions, charge computations, and session state changes. A two-panel layout separates the navigational sidebar from the active working area, a pattern that is common in enterprise-grade web applications precisely because it keeps operational options visible without cluttering the primary workspace.

At the architectural level, the project adheres to the principle of separation of concerns. The presentation layer - HTML, CSS, and JavaScript - has no direct knowledge of how data is stored or retrieved, and the backend - Flask and openpyxl - carries no dependency on how the frontend renders its responses. The API design follows REST conventions as articulated by Fielding (2000), with a POST endpoint dedicated to writing records and a GET endpoint reserved for retrieval.

5.2 Coding Standards

The following practices were maintained consistently across the codebase:

- JavaScript identifiers adhere to camelCase notation (for example, calcBtn, renderReport, updateDateTime), consistent with the conventions established by the JavaScript community.
- Python source files conform to the PEP 8 style guide, including four-space indentation, descriptive variable naming, and logically ordered import statements.
- Each JavaScript function addresses a single, clearly defined responsibility, keeping individual units compact and independently testable without reference to the rest of the application.
- Inline comments accompany logically significant sections of code to clarify intent and reduce the effort required for future maintenance.
- Unused or obsolete code segments were identified and removed ahead of final submission to keep the codebase clean and auditable.
- Input validation is enforced at two distinct levels: HTML5 required attributes and pattern constraints operate on the frontend before any network request is made, while JSON field verification on the backend ensures that no incomplete or malformed record is committed to the workbook.

5.3 Testing Standards

The testing approach draws from the principles set out in IEEE Standard 829 for software and system test documentation. Each test case was assigned a unique identifier and documented with a descriptive title, the specific condition under examination, the system's observed response, and the corresponding expected outcome - as presented in full in Chapter 4. Functional testing was the primary focus, with individual cases covering the principal workflows: authentication, data entry, charge computation, form submission, and report generation.

The suite additionally addresses boundary and edge conditions, including overnight parking and the submission of incomplete forms, to confirm that the system degrades gracefully rather than producing incorrect or misleading outputs when confronted with atypical inputs. This approach ensures that the delivered software behaves predictably across the full range of real-world usage scenarios.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

The Parking Management System developed through this project demonstrates that a thoughtfully assembled combination of accessible web technologies is fully capable of delivering a practical and useful administrative tool. The application removes the dependency on paper-based record-keeping, automates billing calculations that were previously performed by hand, and presents operators with an interface that can be learned and used effectively without any formal training.

The development process offered the team substantive, hands-on experience with full-stack web development, RESTful API design, and file-based data management. Technical challenges along the way - most notably the correct handling of overnight parking edge cases and the resolution of cross-origin communication constraints between the frontend and backend - were worked through via independent research and iterative testing, yielding deeper understanding than classroom instruction alone could provide.

The system as delivered satisfies all core requirements established at the outset of the planning phase. It is fully functional, has been comprehensively tested across all anticipated usage scenarios, and is ready for live deployment subject to two minor adjustments: replacing the hardcoded login credentials with appropriately secured values, and hosting the Flask server on a machine accessible within the institution's local network.

6.2 Future Scope

The current implementation establishes a sound operational foundation. Several enhancements are envisioned for future development iterations:

- Database Integration: Replacing the Excel-based storage layer with a relational database such as SQLite or MySQL would enable concurrent multi-user access, support more sophisticated analytical queries, and scale more reliably as transaction volumes increase.
- Role-Based Access Control: Introducing differentiated permission levels would allow gate operators to be restricted to data entry functions while administrators retain full access to reporting, configuration, and audit capabilities.
- Live Slot Availability Tracking: A real-time occupancy module displaying how many spaces are currently in use would give operators immediate situational awareness and support more proactive congestion management.

- Mobile-Responsive Interface: Adapting the CSS to accommodate smaller viewports would extend the system's usability to tablets and smartphones, broadening the range of devices from which it can be operated.
- Automated Reporting and Notifications: Scheduled report generation with email delivery would eliminate the manual effort currently needed to compile periodic summaries for management, improving operational transparency.
- QR Code or RFID Integration: Coupling the system with a QR scanner or RFID reader at entry and exit points would automate vehicle identification, removing manual registration number entry and reducing the scope for transcription errors.

References

[1] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA: O'Reilly Media, 2018.

[2] E. Freeman and E. Robson, Head First HTML and CSS, 2nd ed. Sebastopol, CA: O'Reilly Media, 2012.

[3] openpyxl Documentation. [Online]. Available: https://openpyxl.readthedocs.io/en/stable/

[4] Flask Documentation. [Online]. Available: https://flask.palletsprojects.com/

[5] Mozilla Developer Network, JavaScript Reference. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[6] IEEE Standard 829-2008, Standard for Software and System Test Documentation. New York, NY: IEEE, 2008.

[7] R. Fielding, Architectural Styles and the Design of Network-based Software Architectures. Ph.D. dissertation, Univ. of California, Irvine, 2000.

[8] Python Software Foundation, Python 3.11 Documentation. [Online]. Available: https://docs.python.org/3.11/

[9] Singla, A. and Kaur, R. (2020). Web-Based Parking Management System for Campus Environments. International Journal of Computer Applications, 175(12), 1-6.

[10] Sharma, P., Verma, A., and Singh, R. (2021). IoT-Enabled Smart Parking System Using RFID and Flask Backend. Proceedings of the IEEE International Conference on Smart Mobility, pp. 112-118.

Individual Contribution

The following pages present individual contribution reports for each member of the project group, detailing their respective roles, technical findings, and specific contributions to the report and presentation.

PARKING MANAGEMENT SYSTEM

SHUBHAM CHAKRABORTY

2305572

Abstract:

This project involved the design and development of a web-accessible Parking Management System for KIIT University campus, replacing a manual paper-based workflow with a structured digital solution. The stack comprises HTML, CSS, and JavaScript on the frontend, Python Flask on the backend, and openpyxl for Excel-based data persistence, together enabling real-time vehicle tracking, automated charge computation, and comprehensive administrative reporting.

Individual Contribution and Findings:

Role: Frontend Developer and UI/UX Designer

- Designed and built the complete frontend interface, including login.html, index.html, style.css, and login.css.
- Developed the real-time vehicle data entry form with field-level validation through HTML5 attributes and JavaScript pattern constraints.
- Implemented the live clock display and dynamic panel transitions between data entry and reporting views without full page reloads.
- Ensured visual consistency and a professional appearance appropriate for institutional deployment across all interface components.
- Conducted usability sessions with fellow students to identify and eliminate interface friction points before final submission.

Individual Contribution to Report Preparation:

Authored Chapter 1 (Introduction) and Chapter 2, Section 2.1 (HTML, CSS, and JavaScript). Contributed to the Abstract and Acknowledgements. Prepared the List of Figures and all figure captions.

Full Signature of Supervisor: _____ Full Signature of Student: _____

PARKING MANAGEMENT SYSTEM

MRINMAY DAS

23051682

Abstract:

This project involved the design and development of a web-accessible Parking Management System for KIIT University campus, replacing a manual paper-based workflow with a structured digital solution. The stack comprises HTML, CSS, and JavaScript on the frontend, Python Flask on the backend, and openpyxl for Excel-based data persistence, together enabling real-time vehicle tracking, automated charge computation, and comprehensive administrative reporting.

Individual Contribution and Findings:

Role: Backend Developer and Data Management Specialist

- Designed and implemented the Flask backend server (app.py), including the /submit POST endpoint and the /excel GET endpoint.
- Integrated the openpyxl library for dynamic workbook creation, row appending, and column header initialisation.
- Configured Flask-CORS to resolve cross-origin resource sharing restrictions between the browser-served frontend and the locally running backend.
- Implemented server-side JSON field validation to enforce data integrity before any write operation is committed to the workbook.
- Conducted direct fetch call testing from the browser console to validate all backend routes independently of the frontend UI.

Individual Contribution to Report Preparation:

Authored Chapter 2 (Sections 2.2 on Python and Flask, 2.3 on openpyxl, and 2.4 on REST API Architecture) and Chapter 5 (Standards Adopted). Contributed technical content on backend architecture to Chapter 3.

Full Signature of Supervisor: _____ Full Signature of Student: _____

PARKING MANAGEMENT SYSTEM

KUMAR SAMARTH

23055545

Abstract:

This project involved the design and development of a web-accessible Parking Management System for KIIT University campus, replacing a manual paper-based workflow with a structured digital solution. The stack comprises HTML, CSS, and JavaScript on the frontend, Python Flask on the backend, and openpyxl for Excel-based data persistence, together enabling real-time vehicle tracking, automated charge computation, and comprehensive administrative reporting.

Individual Contribution and Findings:

Role: Core Logic Developer and Integration Engineer

- Implemented the core JavaScript calculation engine in script.js for computing parking duration and charges, including the overnight parking edge-case correction.
- Developed the localStorage-based session management system for browser-side authentication state persistence.
- Integrated the frontend form submission pipeline with the Flask backend, establishing the full HTTP POST workflow for parking record submission.
- Implemented the reporting panel with date-range filtering and tabular display of retrieved records.
- Led the end-to-end integration testing phase, coordinating between the frontend and backend sub-teams to resolve communication issues.

Individual Contribution to Report Preparation:

Authored Chapter 3 (Problem Statement, Requirement Specifications, and System Design) and Chapter 4 (Sections 4.1 Implementation Overview and 4.3 Result Analysis). Produced the system architecture and data flow diagrams.

Full Signature of Supervisor: _____ Full Signature of Student: _____

PARKING MANAGEMENT SYSTEM

ANKIT SEN

2305518

Abstract:

This project involved the design and development of a web-accessible Parking Management System for KIIT University campus, replacing a manual paper-based workflow with a structured digital solution. The stack comprises HTML, CSS, and JavaScript on the frontend, Python Flask on the backend, and openpyxl for Excel-based data persistence, together enabling real-time vehicle tracking, automated charge computation, and comprehensive administrative reporting.

Individual Contribution and Findings:

Role: Testing Lead and Documentation Coordinator

- Designed the complete structured test plan comprising eight functional test cases spanning standard workflows, boundary conditions, and error scenarios.
- Executed all test cases systematically and recorded outcomes, observed system behaviour, and any deviations from expected results.
- Coordinated the peer code review sessions that identified logical oversights and improved code clarity ahead of final submission.
- Compiled and formatted the final project report, ensuring consistent presentation, appropriate academic tone, and compliance with KIIT project report format guidelines.
- Managed version control and file organisation across the project codebase throughout the full development lifecycle.

Individual Contribution to Report Preparation:

Authored Chapter 4 (Sections 4.2 Testing and Verification Plan, and 4.4 Quality Assurance), Chapter 6 (Conclusion and Future Scope), and the References section. Compiled, proofread, and formatted the complete final project report for submission.

Full Signature of Supervisor: _____ Full Signature of Student: _____

PLAGARISM REPORT

📊 Estimated Plagiarism Risk

- **Low to Moderate overall (10–25%)**
  - Mostly from **generic definitions + technical descriptions**
  - Not from copied project content

📊 Closest Accurate Estimate (Based on Text Analysis)

If your document were run through a standard plagiarism checker, it would most likely return:

Estimated Similarity: ~18%